UNIVERSIDAD TÉCNICA FEDERICO SANTA MARÍA

SEDE CONCEPCIÓN · DEPARTAMENTO DE ELECTRÓNICA E INFORMÁTICA

TALLER DE ADMINISTRACIÓN DE SISTEMAS

Proyecto integrador · Nodo Sur 2026

ENTREGA 1 · INFORME DE DISEÑO Y PLANIFICACIÓN

DATOS DE IDENTIFICACIÓN	
Título del proyecto	Nodo Sur — Plataforma digital resiliente para una red de emprendimientos y experiencias regionales
Equipo / grupo	*6*
Integrantes	*JoseCarlos Nicolas Vidal Quisel, Octavio Vicente Valencia Henriquez, Lukas Alberto Garrido Maldonado.*
Docente	José Antonio Arellano V.
Asignatura	EIN-090B · Taller de Administración de Sistemas
Fecha de entrega	*[10/09/2026]*
Versión del informe	v1.0
Respositorio	https://github.com/LukasGarrido/TAS

Segundo semestre · 2026

Ponderación: 10% de la calificación grupal del proyecto

ÍNDICE DE CONTENIDOS

Actualice esta tabla de contenido en Word: clic derecho > Actualizar campo.

1 INTRODUCCIÓN

Nodo Sur es una red regional que agrupa pequeñas marcas, talleres creativos, productores y operadores de experiencias, quienes actualmente gestionan sus ventas, reservas, inscripciones y atención a clientes mediante las redes sociales, con formularios aislados y mensajería. Actualmente esta modalidad de operación dificulta la centralización de la información, aumenta el riesgo de perdida de datos y genera problemas de disponibilidad de servicios.

El proyecto tiene como fin el diseñar e implementar una plataforma digital, la cual sea *centralizada, administrable* y *resiliente* que permita a distintos miembros de Nodo Sur publicar sus productos y experiencias, gestionar pedidos y reservas, ofrecer contenidos y mantener comunicación con sus clientes **desde un único dominio**, procurando mantener la operación ante la falla de uno de los dos backends web, siempre que el otro pueda absorber la demanda (criterio N-1).

El alcance de esta primera entrega corresponde al levantamiento de requerimientos, la selección de la aplicación base y el diseño de la arquitectura de referencia. Como equipo, ya avanzamos en la parte del trabajo técnico de la Entrega 2 (instalación de WordPress con Nginx y PHP-FPM, configuración de la base de datos y acceso remoto entre VMs, según consta en la bitácora de uso de IA del repositorio), lo cual valida en la práctica varias de las decisiones tomadas en este informe.

Como restricción principal, la organización no cuenta con un equipo de desarrollo permanente, por lo que la solución debe basarse en software libre o de bajo costo y ser administrable por personal con capacitación básica. Además, el tráfico presentará picos breves de demanda durante las campañas de difusión, lo que condiciona directamente las decisiones de disponibilidad y escalabilidad.

Actores, activos y flujos críticos. Los actores son las pequeñas marcas, talleres creativos, productores y operadores de experiencias de la red regional, el personal con capacitación básica que administrará la plataforma, los clientes finales y el equipo del proyecto, que actúa como proveedor de infraestructura. Los activos críticos son el catálogo de productos y servicios, los registros de ventas, reservas e inscripciones, la base de datos separada, los archivos subidos al CMS y las credenciales administrativas. Los flujos de negocio son la venta de catálogo, la inscripción a talleres, las reservas con cupos limitados, la publicación de contenidos y la atención posventa.

Consecuencias de las fallas sobre activos y flujos:

Activo / flujo	Falla	Pérdida o interrupción concreta	Requisito que lo cubre
Base de datos (pedidos, reservas, inscripciones)	Servicio caído o dato perdido	Sin ventas ni reservas mientras dure la falla; pedidos o inscripciones sin registro; cupos sobrevendidos o perdidos	RNF-02, RNF-06

Activo / flujo	Falla	Pérdida o interrupción concreta	Requisito que lo cubre
Archivos subidos (wp-content/uploads)	Divergencia entre backends o disco lleno	Imágenes de catálogo que aparecen o desaparecen según el nodo que atiende; imposibilidad de publicar productos	RNF-03
Nodo perimetral (DNS + Nginx)	Caída del host	Sitio completo inaccesible aunque los backends estén sanos	RNF-01, RNF-05
Backend web (CMS 1 o CMS 2)	Caída de un nodo	Sin impacto si el otro absorbe la demanda; si no la absorbe, degradación o interrupción de ventas durante la campaña	RNF-01
Flujo de reserva con cupos	Escritura duplicada o perdida	Doble reserva o cupo perdido (caso de las solicitudes duplicadas del incidente piloto)	RF-01
Respaldos	Backup antiguo o nunca restaurado	Pérdida de semanas de datos y recuperación incierta (caso del respaldo sin probar)	RNF-06
Credenciales administrativas	Compartidas sin revocación	Acceso indefinido de personas que ya no deberían tenerlo, sin trazabilidad	RNF-04
Monitorización y logs	Ausencia de métricas y alertas	Fallas de horas sin diagnóstico (caso del formulario caído casi dos horas)	RNF-05, RNF-08

1.1 Análisis de Requisitos

Cada requisito incluye un umbral medible y una prueba de validación: instalar un servicio no demuestra por sí solo el cumplimiento.

ID	Requisito verificable	Prioridad	Criterio de aceptación (umbral)	Prueba de validación
RF-01	CMS funcional para catálogo, pedidos, reservas y administración de contenidos	Alta	Un flujo completo (producto o reserva → pedido registrado) se ejecuta bajo el dominio único y el registro es localizable en la aplicación y en la BD	Ejecutar la compra o reserva desde un cliente y consultar la tabla correspondiente en la BD
RNF-01	Disponibilidad del servicio mediante al menos dos nodos web	Alta	Con un backend detenido, ≥ 99 % de 100 solicitudes GET consecutivas responden 200 y la conmutación ocurre en ≤ 30 s (proxy Nginx, originalmente HAProxy; detección pasiva, ADR-002)	Detener Nginx en un backend y lanzar 100 solicitudes con curl en bucle; revisar los logs de acceso del proxy
RNF-02	La base de datos opera de manera independiente	Alta	El puerto 3306 solo responde a 10.33.199.50 y 10.33.199.51; cualquier otro origen es rechazado	nc -zv 10.33.199.53 3306 desde un origen autorizado y desde uno no autorizado; revisión de reglas del firewall
RNF-03	Persistencia y coherencia del estado compartido	Alta	Un archivo subido por un backend se sirve con HTTP 200 y el mismo checksum desde el otro backend en ≤ 5 s	Subir una imagen y descargarla vía ambos backends; comparar sha256sum

ID	Requisito verificable	Prioridad	Criterio de aceptación (umbral)	Prueba de validación
RNF-04	Administración remota solo por canales seguros y nominativos	Media	Autenticación SSH por clave pública, un usuario por integrante, sin login por contraseña ni cuentas compartidas; una cuenta puede revocarse sin afectar a las demás	Revisar sshd_config; revocar una clave de prueba y verificar que el acceso falla
RNF-05	Detección de falla crítica en tiempo acotado	Media	Alerta automática recibida en ≤ 5 min tras detener un backend, sin observar un panel	Detener un servicio y medir el tiempo entre el evento y la recepción de la alerta
RNF-06	Continuidad mediante respaldos verificados	Alta	Retención ≥ 30 días; restauración completa en ≤ 60 min con pérdida de datos ≤ 24 h	Restaurar en una VM de prueba y medir tiempos y datos recuperados (Entrega 4)
RNF-07	Publicación bajo un dominio único con HTTPS válido	Alta	https://tas-06.arpa responde sin error de certificado y HTTP en el puerto 80 redirige a HTTPS	curl -v y openssl s_client desde un cliente
RNF-08	Trazabilidad y hora consistente en logs	Media	Desfase entre nodos ≤ 1 s; un evento puede reconstruirse correlacionando logs de proxy, web, BD y SSH	Comparar chronyc tracking en todos los nodos y seguir una solicitud entre logs

2 RECOMENDACIÓN DE LA APLICACIÓN

Criterio	Alternativa A (Joomla)	Alternativa B (Drupal)	Decisión
Licenciamiento	GPL, gratuito	GPL, gratuito	Empate en costo entre las tres opciones, no es un factor diferenciador
Comunidad/soporte	Grande (tercer CMS más usado a nivel mundial), pero ecosistema de extensiones más pequeño que WordPress	Grande pero orientada a desarrolladores y proyectos enterprise/gobierno, menos enfocada en e-commerce	WordPress, por su ecosistema maduro de plugins de e-commerce y reservas
Curva de operación	Media: requiere entender su estructura de artículos, categorías y menús, más rígida que WordPress	Alta: requiere conocimientos técnicos serios de arquitectura de contenidos, módulos, y en ocasiones Drush/CLI	WordPress, por menor curva para personal sin equipo técnico permanente
Recursos	Pila similar; requisitos comparables a WordPress	Pila similar, pero con Commerce y módulos a medida el consumo y el tuning requeridos son mayores	WordPress: pila PHP + MySQL/MariaDB soportada por Ubuntu 24.04 (PHP 8.3). Con PHP-FPM, 2 vCPU / 2 GB por backend es una base razonable para el volumen esperado, a validar con prueba de carga
Mantenimiento	Actualizaciones mayores más exigentes y extensiones con ciclos de soporte desiguales	Actualizaciones mayores costosas; requiere personal con conocimiento técnico para mantenerlo	WordPress: parches frecuentes del núcleo; el mayor riesgo está en los plugins de terceros. Política: actualizar primero en pruebas, limitar la cantidad de plugins y deshabilitar la edición de archivos desde el panel
Compatibilidad	Compatible con la pila, pero e-commerce y reservas dependen de extensiones poco integradas	Compatible con la pila; e-commerce requiere Drupal Commerce y desarrollo a medida	WordPress: compatible con Nginx + PHP-FPM + MySQL de Ubuntu 24.04. WooCommerce exige una versión mínima de WordPress y PHP; el plugin de reservas debe validarse con dos nodos

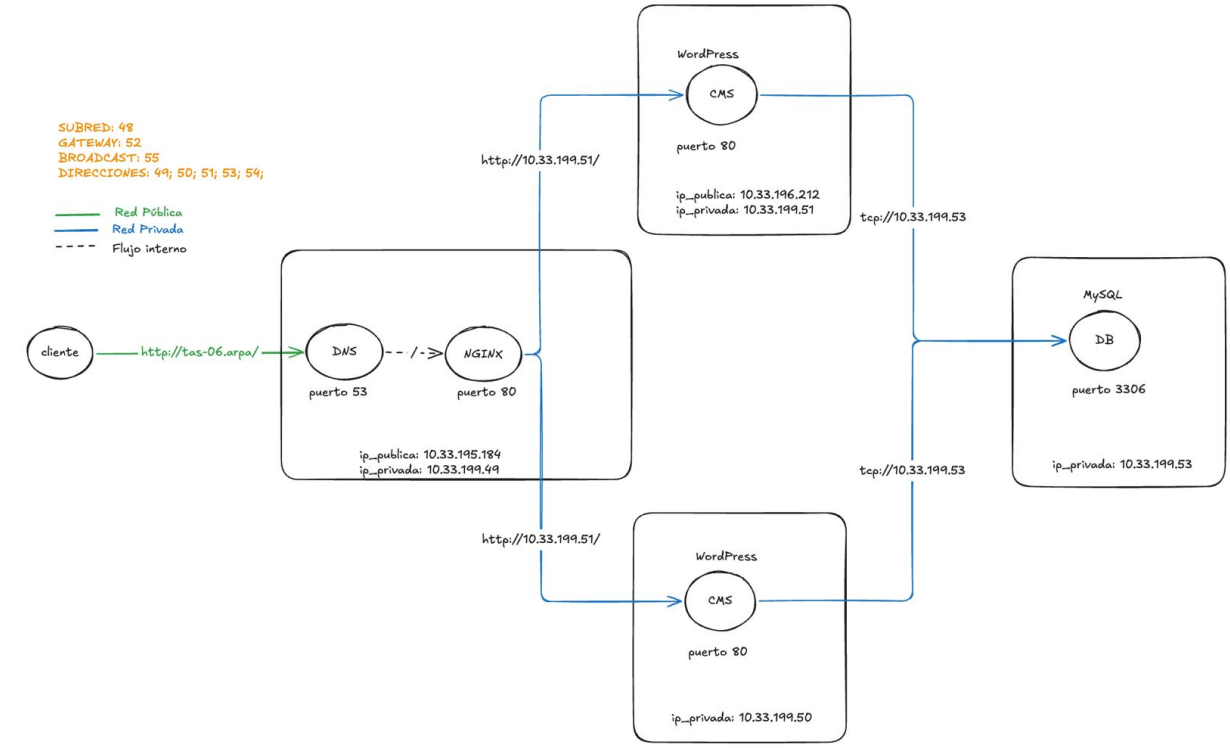
Criterio	Alternativa A (Joomla)	Alternativa B (Drupal)	Decisión
Límites conocidos	No tiene un módulo de e-commerce/reservas tan maduro como WooCommerce; VirtueMart o HikaShop son menos pulidas	No tiene e-commerce/reservas nativo; requiere Drupal Commerce y desarrollo a medida, con mantenimiento más costoso	WordPress, por cubrir catálogo + pedidos + reservas + contenido en un solo stack (WooCommerce + plugin de reservas). Sus límites: es stateful (uploads en disco), por lo que con dos nodos exige resolver la coherencia de archivos; wp-cron se dispara por visita y con dos nodos se ejecuta por duplicado (se resuelve con un cron del sistema en un solo nodo); admin-ajax carga bajo tráfico alto; rendimiento sensible a plugins, requiere caché

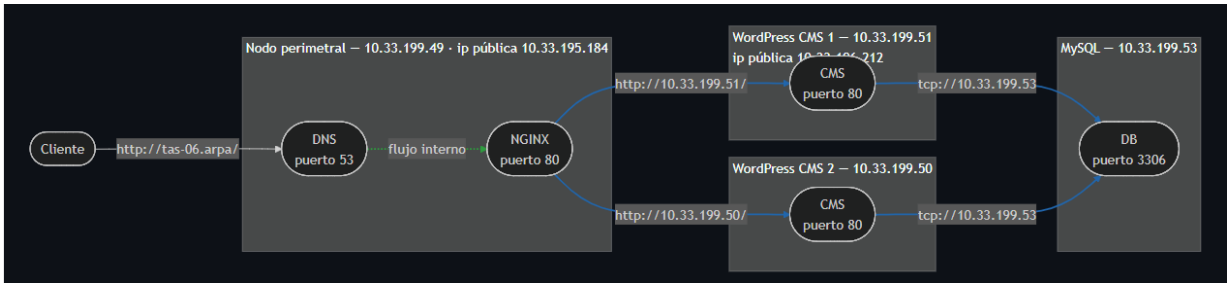
Se evaluaron tres alternativas para el caso de negocio: WordPress, Joomla y Drupal. La aplicación seleccionada es WordPress junto con WooCommerce , servida por Nginx. Joomla se descarta porque, si bien es GPL y tiene una comunidad grande, requeriría sumar extensiones de terceros poco integradas entre sí (como VirtueMart o HikaShop) para cubrir lo que se requeria, agregando fragilidad sin un beneficio real sobre WordPress. Drupal se descarta porque está pensado para sitios complejos con lógica de contenido avanzada, no para una tienda con reservas simple, y exigiría un equipo de desarrollo dedicado que la organización no tiene.

WordPress + WooCommerce cubre en un solo stack el catálogo, los pedidos, la gestión de contenidos y con un plugin adicional de reservas, a definir en la Entrega 2 las reservas de talleres con cupos. Al ser PHP + MySQL, es compatible con la arquitectura de referencia del enunciado, y Nginx ofrece mejor manejo de conexiones concurrentes que alternativas como Apache, lo cual es relevante dado que el caso de negocio anticipa picos de tráfico durante campañas de difusión.

Las versiones exactas de requisitos de cada CMS deben confirmarse en su documentación oficial antes de la entrega final. El riesgo asumido con WordPress es la dependencia del ecosistema de plugins y del trabajo de mantenerlos actualizados, que se mitiga con la política de mantenimiento indicada en la tabla.

3 ARQUITECTURA DE SERVICIOS





La arquitectura implementada (Figura 1) usa una subred /29 (10.33.199.48/29) con dominio interno tas-06.arpa. El cliente resuelve el dominio y accede por HTTPS (puerto 443; el puerto 80 solo redirige a HTTPS) al nodo perimetral (10.33.199.49, IP pública 10.33.195.184), donde DNS y Nginx corren en el mismo host: Nginx recibe la solicitud y la distribuye hacia dos backends WordPress.

Ambos backends (CMS 1 en 10.33.199.51 y CMS 2 en 10.33.199.50) se conectan por red privada a una base de datos MySQL independiente (10.33.199.53), accesible únicamente desde esas dos IPs. Esta decisión de usar Nginx como balanceador en lugar de HAProxy está documentada en el ADR-002 del repositorio: al no contar la versión libre de Nginx con health checks activos, la detección de fallas es pasiva (se detecta tras una solicitud fallida, no antes).

El diagrama muestra que actualmente no existe un componente de storage compartido entre ambos backends WordPress. Esto significa que RNF-03 (persistencia y coherencia de archivos, imágenes y sesiones entre nodos) aún no tiene una solución de arquitectura definida: si un cliente sube una imagen al catálogo a través del CMS 1, ese archivo no estará automáticamente disponible si la siguiente solicitud es atendida por el CMS 2. Esta brecha queda declarada como riesgo abierto para resolver antes de completar la Entrega 3.

Esta brecha se resuelve con un nodo de archivos NFS (10.33.199.54) a incorporar en la Entrega 3, según el ADR-003 (propuesta, por confirmar).

Adicionalmente, WordPress CMS 1 mantiene una IP pública propia (10.33.196.212) además de su IP privada. Esto permite que un cliente acceda directamente a ese backend saltándose el balanceo de Nginx, lo cual contradice el objetivo de un único punto de entrada y debe corregirse en la Entrega 2. Además de retirar esa IP pública, el puerto 80 de ambos backends se restringe por firewall al nodo perimetral (ver «Acceso directo a backends»).

Figura 1. Arquitectura de servicios propuesta. Fuente: elaboración propia.

Servicio / rol	Ubicación	Puerto o flujo	Dependencias
DNS + Nginx (proxy/balanceador)	Nodo perimetral — 10.33.199.49 (pública 10.33.195.184)	53/UDP-TCP DNS; 443/TCP HTTPS; 80/TCP solo redirección a HTTPS	WordPress CMS 1, WordPress CMS 2
WordPress CMS 1 (Nginx + PHP-FPM)	10.33.199.51 (pública 10.33.196.212 — a corregir)	80/TCP (solo desde .49)	MySQL DB, archivos compartidos
WordPress CMS 2 (Nginx + PHP-FPM)	10.33.199.50	80/TCP (solo desde .49)	MySQL DB, archivos compartidos
MySQL DB	10.33.199.53	3306/TCP, solo desde .50 y .51	WordPress CMS 1, WordPress CMS 2
Administración remota (SSH)	Todos los nodos	22/TCP, solo desde la red de administración	Administrador
Storage compartido (NFS) / monitoreo	NFS en 10.33.199.54, a incorporar en E3 (ADR-003); monitoreo pendiente de diseño	NFS 2049/TCP, solo desde .50 y .51	Consumido por CMS 1 y CMS 2 (E3); monitoreo en Entrega 4

Coherencia de archivos entre nodos. Varios nodos web pueden entregar contenidos distintos si no se define dónde vive cada tipo de estado. WordPress se separa así:

Estado	Dónde vive	Cómo se mantiene coherente
Sesiones de WooCommerce, carritos, transients	Base de datos	Ya es coherente entre nodos porque ambos backends usan la misma BD
Archivos subidos (wp-content/uploads)	Disco	Montaje NFS compartido desde .54 (propuesta, ver ADR-003)

Estado	Dónde vive	Cómo se mantiene coherente
Código, plugins, temas y wp-config.php	Disco de cada backend	Despliegue desde archivos versionados y DISALLOW_FILE_MODS para impedir instalar plugins en un solo nodo
Tareas programadas (wp-cron)	Cada backend	DISABLE_WP_CRON en ambos y un cron del sistema en un solo nodo

La dependencia de archivos se resuelve con NFS por simplicidad y coherencia inmediata. La alternativa evaluada (lsyncd/rsync entre backends) evita un nodo nuevo, pero introduce una ventana de divergencia de segundos. El nodo NFS pasa a ser un punto único de falla. No se exige NFS como tecnología: se exige resolver esta dependencia y verificarla con la prueba de checksum de RNF-03. La decisión completa se documenta en el ADR-003.

Alcance de los health checks. La versión libre de Nginx solo ofrece detección pasiva (max_fails, fail_timeout y proxy_next_upstream con error, timeout, http_502, http_503 y http_504), según el ADR-002. Esto delimita qué fallas detectan las comprobaciones propuestas:

Detecta	No detecta
Servicio caído o conexión rechazada	Contenido distinto entre nodos
Timeouts del backend	Respuestas 200 lentas pero bajo el timeout
Respuestas 502, 503 y 504	Caída de la BD (afecta a ambos backends por igual)

Nginx tampoco reintenta por defecto solicitudes no idempotentes (POST): un pedido que falla a mitad de camino no se reenvía al otro backend, de modo que se evita duplicarlo pero ese cliente ve el error. Como complemento se propone un script con curl a un endpoint de salud de cada backend (que verifica también la conexión a la BD), ejecutado por cron y con alerta, para dar detección activa fuera de Nginx.

Acceso directo a backends. Los backends no deben poder eludir los controles del punto de entrada:

- El puerto 80 de CMS 1 y CMS 2 acepta conexiones únicamente desde 10.33.199.49 (firewall en cada backend), y el SSH solo desde la red de administración.
- Se retira la IP pública del CMS 1 (Entrega 2).
- Nginx incluye un server por defecto que rechaza Hosts desconocidos, y los backends usan set_real_ip_from 10.33.199.49 para registrar la IP real del cliente.
- Prueba de aceptación: curl http://10.33.196.212 desde fuera debe fallar, y curl http://10.33.199.51 desde un origen distinto de .49 debe ser rechazado.

El tráfico entre el nodo perimetral y los backends viaja sin cifrar dentro de la subred privada; se acepta como riesgo residual para este alcance.

4 CONSIDERACIONES DE RECURSOS Y ESCALABILIDAD

Componente	CPU / RAM	Almacenamiento	Supuesto de carga	Evolución
Nodo perimetral (DNS + Nginx)	1 vCPU / 1 GB RAM	10 GB	Bajo uso de CPU en régimen normal; concentra DNS y balanceo en un solo host	Separar DNS y Nginx en nodos distintos para reducir el SPOF más crítico de la arquitectura actual
WordPress CMS 1	2 vCPU / 2 GB RAM	20 GB	Picos breves de tráfico durante campañas de difusión	Retirar su IP pública directa y dejar el nodo perimetral como único punto de entrada
WordPress CMS 2	2 vCPU / 2 GB RAM	20 GB	Igual que CMS 1, en espejo	Mantener como réplica funcional equivalente a CMS 1
MySQL DB	2 vCPU / 4 GB RAM	30 GB	Consultas concentradas en horarios de campaña	Incorporar réplica maestro-esclavo en entregas posteriores
Storage compartido (pendiente)	No implementado	No implementado	Necesario para que wp-content sea coherente entre CMS 1 y CMS 2	Diseñar e implementar en Entrega 3 (NFS u objeto compatible S3) para cerrar RNF-03
Monitoreo y backup (pendiente)	No implementado	No implementado	Actualmente solo existe un script base de respaldo de la BD (octavio_bd_backup.sql)	Definir e implementar en Entrega 4: métricas, alertas y backups automatizados de archivos y configuración

Puntos únicos de falla identificados: el nodo perimetral, que concentra DNS y Nginx en un mismo host su caída interrumpe simultáneamente la resolución del dominio y la distribución de tráfico, la base de datos, que es una instancia única sin réplica; y la ausencia de storage compartido, que impide garantizar RNF-03 mientras no se implemente. Cuando se incorpore el nodo de archivos NFS (10.33.199.54), este pasa a ser también un punto único de falla: su caída deja sin uploads a ambos backends. Todos los nodos dependen además de la infraestructura de virtualización y red de laboratorio, que el equipo no controla.

Es importante diferenciar los conceptos: el balanceo de carga (Nginx distribuyendo tráfico entre dos backends) no equivale por sí solo a alta disponibilidad, ya que el nodo perimetral sigue siendo un punto único de falla y, al usar la versión libre de Nginx, la detección de fallas es pasiva en vez de activa (ver ADR-002). La alta disponibilidad completa requeriría separar DNS de Nginx, replicar la base de datos y definir la persistencia del estado compartido evolución planteada para entregas posteriores. En términos de servicio, la tolerancia a fallas de la capa web (la caída de un backend, si el otro tiene capacidad) no equivale a la

disponibilidad del servicio completo, que además depende del nodo perimetral, la base de datos y el nodo de archivos.

Capacidad con un solo backend (criterio N-1). Dos nodos no garantizan continuidad si uno no puede absorber la demanda durante la falla del otro. El criterio de diseño es que cada backend debe poder atender el 100 % del pico previsto de campaña.

- Medición: prueba de carga (k6 o ab) sobre la página de catálogo y el checkout, con un solo backend activo.
- Umbrales de aceptación: p95 ≤ 2 s, CPU ≤ 85 % y errores < 1 %.
- Condición: pico proyectado de campaña ≤ capacidad medida de un backend.

Cálculo de referencia (supuestos ilustrativos, que deben reemplazarse con las mediciones):

Parámetro	Supuesto	Resultado
Workers PHP-FPM por backend	(2048 MB − ~600 MB para SO y Nginx) ÷ ~60 MB por worker	≈ 24; se fija pm.max_children ≈ 20
Capacidad teórica por backend	20 workers ÷ 0,3 s por solicitud	≈ 65 req/s
Capacidad con plugins reales	50 % de la teórica	≈ 30 req/s por backend
Conexiones a la BD	2 backends × 20 workers + margen	max_connections ≥ 50

Si el pico proyectado supera la capacidad de un backend, se aplican medidas de degradación controlada: caché FastCGI en Nginx, limit_req para proteger el checkout y suspensión de tareas no críticas.

Objetivos de disponibilidad. La meta es una disponibilidad mensual ≥ 99,5 %, equivalente a un presupuesto de unas 3,6 h de indisponibilidad al mes (0,5 % de 720 h), con detección de falla ≤ 5 min, RTO del servicio web ≤ 60 min y RPO ≤ 24 h. Con un RTO de 60 min caben hasta tres incidentes de este tipo al mes, y un cuarto agota el presupuesto.

5 USABILIDAD

Los clientes finales acceden a la plataforma mediante el dominio interno tas-06.arpa, sin necesidad de recordar direcciones IP ni distinguir qué backend los atiende. WordPress ofrece una navegación familiar de catálogo, carrito y confirmación de pedidos o reservas, con mensajes de error estándar y consistentes entre ambos backends.

El personal de Nodo Sur, sin formación técnica avanzada, administra la plataforma desde el panel de administración de WordPress, donde puede publicar contenido, gestionar el catálogo y revisar pedidos o reservas, sin necesidad de acceder a la infraestructura subyacente para sus tareas cotidianas.

Las tareas administrativas de infraestructura quedan reservadas al equipo del proyecto mediante acceso SSH nominativo por clave pública hacia las IP privadas de cada nodo, evitando que el personal de negocio necesite privilegios elevados para operar el sitio.

Mientras no se resuelva la persistencia de archivos compartidos entre backends (ver sección de Consideraciones de Recursos), existe el riesgo de que un cliente perciba inconsistencias , por ejemplo, una imagen de catálogo visible desde un backend pero no desde el otro. Corregir esta brecha es una condición de usabilidad antes de escalar a dos backends.

6 RECOMENDACIONES DE SEGURIDAD Y MONITOREO

Riesgo	Control recomendado	Métrica / log	Validación futura
Credenciales compartidas por mensajería, sin revocación individual	Acceso SSH nominativo por clave pública hacia las IP privadas de cada nodo, sin cuentas compartidas	Logs de autenticación SSH (auth.log)	Entrega 2: verificar acceso nominativo y revocar una cuenta de prueba
Base de datos expuesta a redes no autorizadas	Firewall restringiendo el puerto 3306 solo a las IPs .50 y .51	Logs de conexión de la BD, reglas de firewall aplicadas	Entrega 2/3: nc -zv al puerto 3306 desde un origen autorizado y uno no autorizado; el segundo debe ser rechazado
WordPress CMS 1 accesible directamente por IP pública, saltándose el balanceador	Retirar la IP pública del CMS 1 y limitar el puerto 80 de ambos backends a .49 con firewall, con un server por defecto en Nginx; el nodo perimetral queda como único punto de entrada	Registro de intentos de acceso directo por IP en logs de Nginx del backend	Entrega 2: curl http://10.33.196.212 desde fuera debe fallar y curl http://10.33.199.51 desde un origen distinto de .49 debe ser rechazado
Persistencia de archivos no resuelta entre CMS 1 y CMS 2 (sin storage compartido)	Diseñar e implementar storage compartido (NFS u objeto compatible S3) antes de operar ambos backends en producción	Comparación de archivos/checksum entre backends	Entrega 3: subir un archivo y comparar sha256sum desde ambos backends (mismo checksum en ≤ 5 s)
Falla no detectada a tiempo (como el incidente de 2 horas sin diagnóstico)	Monitorización de Nginx, backends y BD con alertas automáticas, compensar la falta de health checks activos de Nginx libre (ADR-002)	Métricas de disponibilidad, latencia y errores	Entrega 4: detener un servicio y verificar alerta recibida en ≤ 5 min
Backup nunca probado o restauración fallida	Backups automatizados de BD (a partir del script base octavio_bd_backup.sql), archivos y configuración con retención ≥30 días	Inventario de respaldos y checksum de verificación	Entrega 4: restauración en VM de prueba en ≤ 60 min con pérdida de datos ≤ 24 h
Certificado TLS vencido o mal configurado	Certificado validado y renovación gestionada en el nodo perimetral	Logs de Nginx; verificación con curl/openssl	Entrega 2/3: validación de HTTPS desde un cliente externo

Riesgo	Control recomendado	Métrica / log	Validación futura
Un backend no puede absorber el pico cuando cae el otro	Criterio N-1 con prueba de carga; caché FastCGI, limit_req y degradación controlada	p95, CPU y tasa de errores con un solo backend activo	Entrega 3: p95 ≤ 2 s, CPU ≤ 85 % y errores < 1 % con un solo backend

Política de datos. Ubicación, respaldo, retención, acceso y capacidad de cada tipo de dato:

Dato	Ubicación	Respaldo	Retención	Acceso	Capacidad y permisos
Pedidos, reservas, inscripciones y datos de clientes	BD (.53)	Dump diario + copia separada	≥ 30 días	Solo el usuario de la aplicación desde .50 y .51	Alerta al 80 % de disco
Archivos subidos	wp-content/uploads (NFS, .54)	Copia diaria	≥ 30 días	www-data; nunca chmod 777	Límite de tamaño por archivo y alerta al 80 % de disco
Logs	Cada nodo	Centralización en la Entrega 4	A definir (propuesta: 90 días)	Solo administradores nominativos	logrotate
Configuración	Repositorio (saneada)	Git	Historial completo	Cuentas nominativas	Sin secretos

Los datos de clientes son datos personales. El equipo debe verificar con el docente la normativa chilena aplicable (Ley 19.628 y su actualización, Ley 21.719) y no incluir datos personales en el repositorio ni en capturas. Las contraseñas y credenciales se mantienen fuera del repositorio.

7 PLAN DE IMPLEMENTACIÓN, PRUEBAS Y REVERSIÓN

Los responsables se proponen según la distribución actual del trabajo y deben confirmarse en equipo. Una etapa avanza solo si se cumple su criterio; de lo contrario se aplica la reversión indicada.

Etapa	Entregable y responsables	Depende de	Pruebas previstas	Criterio para avanzar	Reversión
Entrega 2	WordPress + Nginx + PHP-FPM + MySQL en VMs separadas, DNS, HTTPS, firewall y retiro de la IP pública del CMS 1. **Responsables:** Octavio (CMS), Lukas (BD), Josecarlos (DNS y nodo perimetral)	VMs y red listas	Resolución DNS de tas-06.arpa, acceso HTTPS por nombre, flujo de compra o reserva, BD inaccesible desde fuera	RNF-02, RNF-07 y RF-01 verificados	Snapshot de VM y copia de nginx.conf antes de cada cambio; nginx -t antes de reload
Entrega 3	Segundo backend, balanceo con upstream, nodo de archivos compartidos y script de salud. **Responsables:** Octavio, Lukas, Josecarlos	Entrega 2 y ADR-003	Distribución entre CMS 1 y CMS 2, retiro de un backend, checksum entre nodos, prueba de capacidad N-1	RNF-01 y RNF-03 verificados y capacidad de un backend ≥ pico proyectado	Quitar el segundo backend del upstream y volver a un solo nodo
Entrega 4	Monitoreo, alertas y backups automatizados con restauración medida. **Responsables:** equipo completo	Entrega 3	Degradación controlada, alerta ≤ 5 min, restauración ≤ 60 min en VM de prueba	RNF-05 y RNF-06 verificados con marcas de tiempo	Restaurar la configuración anterior desde Git

Etapa	Entregable y responsables	Depende de	Pruebas previstas	Criterio para avanzar	Reversión
Entrega 5	Auditoría final e incidente no anunciado. **Responsables:** equipo completo	Entrega 4	Recuperación validada con el flujo de negocio, defensa individual	Matriz de pruebas completa	Etiqueta de versión estable del repositorio

8 CONCLUSIONES

La propuesta es viable dentro de las restricciones del caso, ya que puede implementarse con software libre (WordPress, Nginx, MySQL/MariaDB) y ser administrada por personal sin formación técnica avanzada. Parte de la implementación ya está en curso según la bitácora de uso de IA del repositorio (base de datos migrada entre VMs, acceso remoto habilitado y resolución DNS diagnosticada), lo que valida en la práctica varias de las decisiones de este informe.

Las decisiones prioritarias para avanzar en la Entrega 2 son: retirar la IP pública directa del CMS 1, confirmar el plugin de reservas con cupos que complementará a WooCommerce, y definir el dimensionamiento final de CPU/RAM/disco de cada VM (actualmente estimado, no medido).

El principal riesgo residual reconocido en esta etapa es la concentración de DNS y Nginx en un mismo nodo perimetral, junto con la dependencia de una única instancia de base de datos, su evolución hacia una arquitectura sin puntos únicos de falla queda planteada como trabajo futuro, más allá del alcance de esta entrega. Cabe precisar que el diseño tolera la falla de un backend web, pero eso no equivale a alta disponibilidad del servicio completo.